

Behavioral Patterns - Part 1

Chain of Responsibility Command Iterator Mediator

Sub-Chapter 02-05

Chain of Responsibility	Pass request along a chain until someone handles it
Command	Wrap a request as an object -- execute, queue, undo
Iterator	Traverse a collection without knowing its internal structure
Mediator	Objects talk through a central hub instead of directly

1 Chain of Responsibility

Pass a request along a chain of handlers; each decides to handle or forward it.

What is it?

Chain of Responsibility lets you build a pipeline of handler objects. When a request arrives, it travels along the chain until one handler decides to process it. If a handler cannot deal with the request, it passes the request to the next handler -- so the sender never needs to know which object will ultimately handle it.

Real-world example -- Leave Approval

A TeamLead can approve up to 3 days of leave, a Manager up to 7 days, and a Director approves everything else. Each approver either handles the request or passes it up the hierarchy.

```
abstract class Approver {
    protected Approver next;
    Approver setNext(Approver n) { next=n; return n; }
    abstract void approve(String emp, int days);
}
class TeamLead extends Approver {
    public void approve(String emp, int days) {
        if (days<=3) System.out.println("TeamLead approved "+days+" days for "+emp);
        else if (next!=null) { System.out.println("TeamLead passing up..."); next.approve(emp,days); }
    }
}
class Manager extends Approver {
    public void approve(String emp, int days) {
        if (days<=7) System.out.println("Manager approved "+days+" days for "+emp);
        else if (next!=null) { System.out.println("Manager passing up..."); next.approve(emp,days); }
    }
}
class Director extends Approver {
    public void approve(String emp, int days) {
        System.out.println("Director approved "+days+" days for "+emp);
    }
}
// Build chain and use
Approver lead = new TeamLead();
lead.setNext(new Manager()).setNext(new Director());
lead.approve("Alice", 2); // TeamLead approves
lead.approve("Bob", 5); // Manager approves
lead.approve("Carol", 10); // Director approves
```



Fig 1 -- Request travels left-to-right; each node either handles or forwards it.

Key Rules

- Each handler decides independently whether to handle or forward.
- Handlers are loosely coupled -- adding or removing a handler does not change the sender.
- A request may go unhandled if no handler in the chain accepts it (handle this case).

2 Command

Wrap a request as a self-contained object -- supports execute, queue, log, and undo.

What is it?

The Command pattern encapsulates a request (and all the information needed to carry it out) inside an object. This decouples the object that issues a request (Invoker) from the object that knows how to perform it (Receiver). Because commands are objects, they can be stored in a history stack, queued, or undone.

Real-world example -- Order System

An Invoker executes PlaceOrderCommand objects. Each command wraps the OrderService call and knows how to undo itself (cancel the order).

```
interface Command { void execute(); void undo(); }
class OrderService {
    void placeOrder(String item) { System.out.println("Placed: " + item); }
    void cancelOrder(String item) { System.out.println("Cancelled: " + item); }
}
class PlaceOrderCommand implements Command {
    private OrderService svc; private String item;
    PlaceOrderCommand(OrderService svc, String item) { this.svc=svc; this.item=item; }
    public void execute() { svc.placeOrder(item); }
    public void undo() { svc.cancelOrder(item); }
}
class Invoker {
    private Deque<Command> history = new ArrayDeque<>();
    void run(Command cmd) { cmd.execute(); history.push(cmd); }
    void undo() { if (!history.isEmpty()) history.pop().undo(); }
}
// Usage
OrderService svc = new OrderService();
Invoker invoker = new Invoker();
invoker.run(new PlaceOrderCommand(svc, "iPhone"));
invoker.run(new PlaceOrderCommand(svc, "MacBook"));
invoker.undo(); // Cancelled: MacBook
```



Fig 2 -- Invoker delegates to a Command object; the Command calls the real Receiver.

Key Rules

- The Command interface always has execute() and (optionally) undo().
- Invoker is decoupled from the Receiver -- it only knows about Command.
- A history stack (Deque/Stack) enables multi-level undo.
- Commands can be serialised, queued, or scheduled for later execution.

3 Iterator

Provide a standard way to traverse a collection without exposing its internals.

What is it?

The Iterator pattern gives clients a uniform interface (hasNext / next) to walk through any collection -- array, linked list, tree, etc. -- without knowing how the collection is stored internally. Java's enhanced for-loop and java.util.Iterator are the canonical built-in examples.

Real-world example -- Custom Playlist

A Playlist stores songs in a plain array. The iterator hides the array index arithmetic; client code just calls hasNext() and next().

```
interface Iterator<T> { boolean hasNext(); T next(); }
class Playlist {
    private String[] songs;
    private int size = 0;
    Playlist(int cap) { songs = new String[cap]; }
    void add(String song) { songs[size++] = song; }
    Iterator<String> iterator() {
        return new Iterator<String>() {
            int i = 0;
            public boolean hasNext() { return i < size; }
            public String next() { return songs[i++]; }
        };
    }
}
// Usage
Playlist pl = new Playlist(10);
pl.add("Song A"); pl.add("Song B"); pl.add("Song C");
Iterator<String> it = pl.iterator();
while (it.hasNext()) System.out.println(it.next());
// Song A, Song B, Song C
```

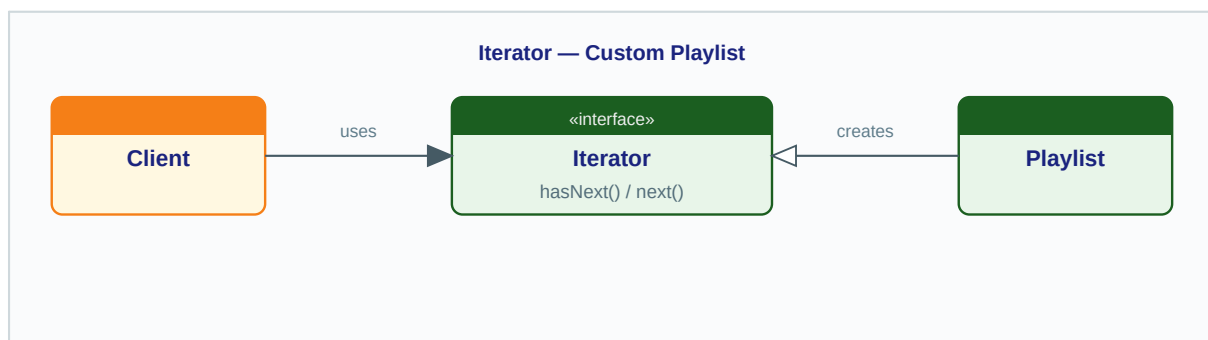


Fig 3 -- Client uses the Iterator interface; Playlist creates and returns a concrete iterator.

Key Rules

- The Client depends only on the Iterator interface, not the concrete collection.
- Multiple independent iterators can traverse the same collection simultaneously.
- Prefer java.util.Iterator / Iterable in production code; roll your own only for custom structures.

4 Mediator

Centralise communication between objects so they do not reference each other directly.

What is it?

The Mediator pattern introduces a central coordinator object. Instead of components talking to each other directly (creating a tangled web of dependencies), all communication is routed through the Mediator. This reduces coupling and makes it easy to change how components interact without touching them individually.

Real-world example -- Chat Room

Users send messages through a ChatRoom (Mediator). Alice never holds a reference to Bob and vice-versa -- the ChatRoom handles routing.

```
class ChatRoom {
    void sendMessage(String from, String to, String msg) {
        System.out.println "[" + from + " -> " + to + "]: " + msg);
    }
}
class User {
    String name; ChatRoom room;
    User(String name, ChatRoom room) { this.name=name; this.room=room; }
    void send(String to, String msg) { room.sendMessage(name, to, msg); }
}
// Usage
ChatRoom room = new ChatRoom();
User alice = new User("Alice", room);
User bob = new User("Bob", room);
alice.send("Bob", "Hi Bob!"); // [Alice -> Bob]: Hi Bob!
bob.send("Alice", "Hey Alice!"); // [Bob -> Alice]: Hey Alice!
```

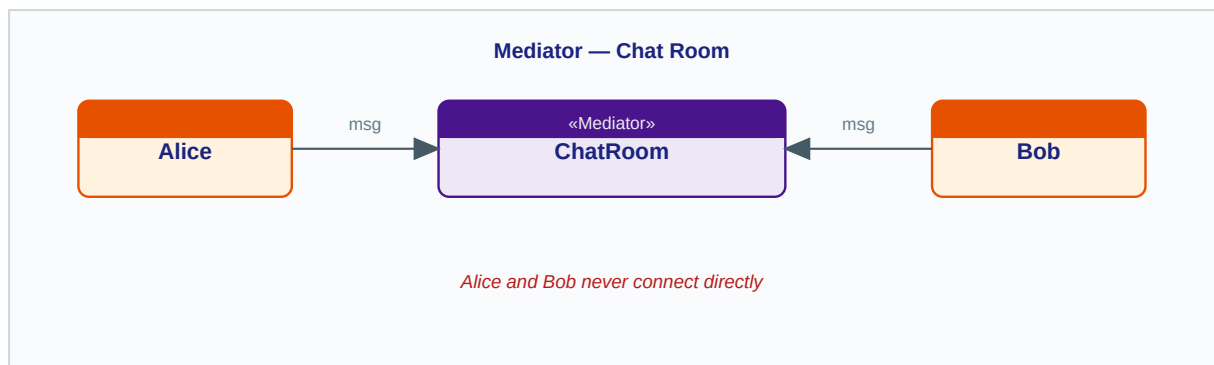


Fig 4 -- Alice and Bob communicate only via ChatRoom; no direct peer-to-peer references.

Key Rules

- Colleagues (Alice, Bob) hold a reference to the Mediator only -- never to each other.
- Adding a new participant means adding a new User; no existing classes change.
- Watch out: a Mediator that grows too large becomes a 'God Object'.

Quick-Reference Cheat Sheet

Pattern	Intent	Key class/method	Real example	Spot violation
Chain of Responsibility	Pass request along a handler chain	Approver.setNext() approve()	Leave approval pipeline	Sender knows which handler will process it
Command	Wrap request as an object; supports undo/queue	Command.execute()) undo() Invoker	Order system with undo	Invoker directly calls Receiver method
Iterator	Traverse collection without exposing internals	hasNext() next()	Custom Playlist iterator	Client accesses internal array index directly
Mediator	Central hub routes messages between components	ChatRoom .sendMessage()	Chat room, Air traffic control	Colleagues hold direct references to each other

Quick-Check Questions

Q1. What happens in Chain of Responsibility if no handler processes the request?

The request goes unhandled (falls off the end of the chain). Always add a default catch-all handler at the end, or check for this condition in the calling code.

Q2. How does the Command pattern enable undo?

Each Command object stores enough state to reverse its effect in undo(). An Invoker keeps a history stack; calling undo() pops the last command and calls its undo() method.

Q3. Why use Iterator instead of directly exposing the internal array?

Exposing the array couples the client to the collection's internal structure. Iterator provides a stable interface so the collection can switch from array to linked list without breaking client code.

Q4. How does Mediator reduce coupling compared to direct object references?

Colleagues store only a reference to the Mediator. When Alice wants to message Bob she calls mediator.sendMessage(); she never imports or constructs Bob herself, so Alice and Bob are completely independent of each other.